Efficient Real-Time Fire Surveillance: A Lightweight Motion-Heuristic Skip Module for Accelerated Inference in Indoor Environments

Hoang Van-Ha, Jong Weon Lee, Park Chun-Su

2026.05.13

Abstract

Conventional deep learning (DL)-based fire and smoke detection systems process every frame of a video stream through a computationally intensive model to classify the presence of fire or smoke. However, in surveillance scenarios — particularly indoor environments — video streams frequently contain static, background-only frames devoid of motion or relevant events, rendering per-frame inference redundant and wasteful of computational resources. To address this limitation, this study proposes a novel skip-module mechanism that leverages motion detection to selectively bypass DL inference on non-informative frames in fire and smoke classification pipelines. Evaluation on a large-scale dataset comprising 150 indoor videos demonstrates that the proposed method substantially improves processing throughput while preserving detection performance. Specifically, the skip module enable the system a nearly 30% increase in frames per second (FPS) while incurring a marginal relative recall reduction of less than 1.5% relative to the baseline (recall 94%), demonstrating the feasibility of plug-and-play inference acceleration for real-time fire and smoke surveillance systems.

1 Introduction

Fires and wildfires, if not detected and controlled in their early stages, can quickly escalate, especially under dry and windy conditions, resulting in catastrophic consequences, including loss of life, destruction of property, and damage to natural ecosystems. For instance, in 2017, urban fires and wildfires in California, USA, caused an estimated economic loss of 10 billion USD [1]. More recently, in 2025, a forest fire in Gyeongsang Province, South Korea, burned approximately 90,000 acres, resulting in at least 27 fatalities and the evacuation of nearly 40,000 residents [2]. Automated early-stage fire and smoke detection systems are therefore critical for enabling timely intervention and minimizing damage.

Numerous fire and smoke detection methods leveraging deep learning (DL) have been proposed in recent years [3], [4]. In practical deployments, these systems are commonly applied to CCTV video streams, where DL-based classifiers or object detectors perform inference on each frame individually. Although this frame-wise paradigm is straightforward to implement, it exhibits two key limitations. First, it relies exclusively on spatial information within individual frames, neglecting temporal cues, such as motion, inherent in video data. Second, in surveillance scenarios, particularly in **indoor** environments, consecutive frames often exhibit minimal variation due to static scene content. Applying a DL model indiscriminately to every frame under such conditions is computationally redundant, increasing processing cost and latency without contributing to detection performance. This inefficiency is further compounded by the well-known accuracy-efficiency trade-off in DL: more accurate models are generally more computationally intensive, and redundant frame processing amplifies these demands, rendering real-time performance difficult to achieve.

A common strategy to reduce inference cost is to substitute a heavy, high-accuracy DL model with a lighter alternative; however, this typically degrades detection reliability, an unacceptable compromise in safety-critical applications such as fire and smoke detection. This study takes a complementary approach: rather than replacing the classifier, we introduce a lightweight skip-module that acts as a computational gate, selectively forwarding only frames with significant scene activity to the high-capacity classifier while bypassing static, non-informative frames. As illustrated in Fig. 1, the conventional pipeline processes every frame through the classifier, whereas the proposed pipeline inserts the skip-module upstream to conditionally suppress redundant inference calls.

In particular, our main contributions are summarized as follows:

- **Skip-Module Mechanism:** We propose a lightweight skip-module that exploits motion estimation via frame differencing to identify static scenes and bypass DL inference on static frames, reducing computational cost without modifying the underlying classifier. The module is designed as a plug-and-play component compatible with existing fire and smoke detection pipelines. To identify the optimal operating configuration, we further develop a systematic hyperparameter optimization procedure that maximizes throughput while preserving detection performance.
- **Indoor Fire and Smoke Video Dataset:** We construct an annotated dataset comprising 150 indoor fire and smoke videos (including fire, smoke, and background-only classes) captured by static surveillance cameras at resolutions ranging from 814×720 to 3840×2160. The dataset covers diverse environments such as warehouses, parking areas, and offices under varying lighting conditions, providing a realistic benchmark for evaluating detection systems in static surveillance scenarios.
- **Comprehensive System Evaluation:** We integrate the skip-module with a high-capacity DL classifier (BIG model) and evaluate the combined system on our video dataset against the baseline (BIG model without skipping) and existing

methods. Results demonstrate a 30% improvement in FPS with a recall reduction of less than 1%, alongside an ablation study that provides insights into system performance and limitations.

The rest of this paper is organized as follows: Section 2 reviews existing fire and smoke detection methods for video and relevant motion detection techniques, contextualizing the need for efficient processing in static surveillance scenarios. Section 3 describes the proposed system architecture, the design of the skip-module, its integration with the BIG model, and the hyperparameter optimization strategy. Section 4 presents the experimental setup, evaluation metrics, and performance results on our large-scale indoor video dataset. Finally, Section 5 summarizes the key findings and discusses limitations and future research directions.

2 Related Work

Fire and Smoke Detection in Images and Videos: Automated fire and smoke detection has attracted considerable research interest, with deep learning emerging as the dominant paradigm. The work [3] provides a comprehensive survey of visual fire detection methods, highlighting the rapid adoption of DL-based approaches in this domain. The majority of these methods formulate the problem as image-level binary classification (fire/smoke vs. none) or object detection, in which a model receives a single RGB image and outputs either a class label with confidence score or localized bounding boxes [5], [6]. Research efforts have primarily focused on improving model accuracy and inference speed through architectural modifications and advanced training strategies. In video-based deployments, these image classifiers are applied directly to each frame, enabling integration with existing pipelines without architectural changes.

Relying exclusively on individual frames, however, discards temporal information inherent in video data that is potentially informative for detection. [7] survey video-based fire and smoke detection approaches and highlight the benefit of modeling temporal dynamics. Representative methods include the work of [8], who combine a CNN for spatial feature extraction with a bidirectional LSTM for temporal modeling, and [9], who employ 3D CNNs with attention mechanisms to jointly capture spatio-temporal patterns. While these approaches demonstrate improved detection performance over purely image-based methods, they incur greater computational cost and require more laborious dataset construction involving temporal annotation. Hybrid methods address this partially by combining image-based classifiers with lightweight temporal post-processing — such as temporal voting [10] or motion-based false positive suppression [11] — yet still apply DL inference unconditionally to every frame, regardless of scene content.

Across both paradigms, the dominant deployment pattern applies DL inference to each frame or short clip, without considering whether inference is warranted given the frame content. In surveillance scenarios, particularly **indoors**, video streams frequently consist of purely background frames with no motion, in which fire or smoke is a priori unlikely. Motivated by this observation, we propose a skip-module that filters such low-activity frames prior to DL inference, reducing computational cost while with minimal degradation in detection reliability. The module prioritizes retaining positive frames (fire/smoke present) while skipping negative frames (background only), functioning as a complementary accelerator for any existing frame-wise detection pipeline.

Motion Detection and the Proposed Skip Module: The skip decision in our module relies on motion estimation, which connects to the well-studied problem of background subtraction. [12] and [13] provide comprehensive reviews of background subtraction methods, ranging from simple statistical models to deep learning-based approaches. More robust methods such as MOG2 and KNN [14] model the scene background statistically and are resilient to gradual illumination changes; however, they introduce non-trivial memory overhead and initialization latency. As the skip-module is designed primarily as a fast, lightweight gate, we instead adopt motion estimation based on frame differencing, which offers minimal computational overhead and requires no background model initialization.

Existing efficient video inference methods, such as FrameExit [15], employ learned gating mechanisms to selectively process frames based on prediction confidence, thereby reducing redundant inference; however, they require joint training of the gating mechanism and the underlying classifier, limiting plug-and-play applicability to pre-trained models. In contrast, this study adopts a training-free approach by integrating lightweight frame-differencing-based motion estimation as a plug-and-play gate, requiring no retraining of the underlying classifier, as described in Section 3.

3 The proposed method

3.1 System Overview

The proposed system is a fire and smoke detection pipeline that processes a continuous video stream and classifies each frame as either containing fire/smoke or not. The core idea is to augment the conventional frame-wise inference pipeline with a lightweight skip module $\mathcal{S}$, which serves as a gating mechanism that selectively suppresses DL inference on frames with low likelihood of containing fire or smoke — most commonly, static background frames — thereby reducing redundant computation and improving overall throughput. Formally, let f_t denote the video frame at time step t, $\mathcal{M}$ the DL classifier, and $\hat{y}_t \in 0, 1$ the predicted label for f_t.

Baseline system. Each frame is directly passed to the classifier:

$$\hat{y}_t = \mathcal{M}(f_t)$$

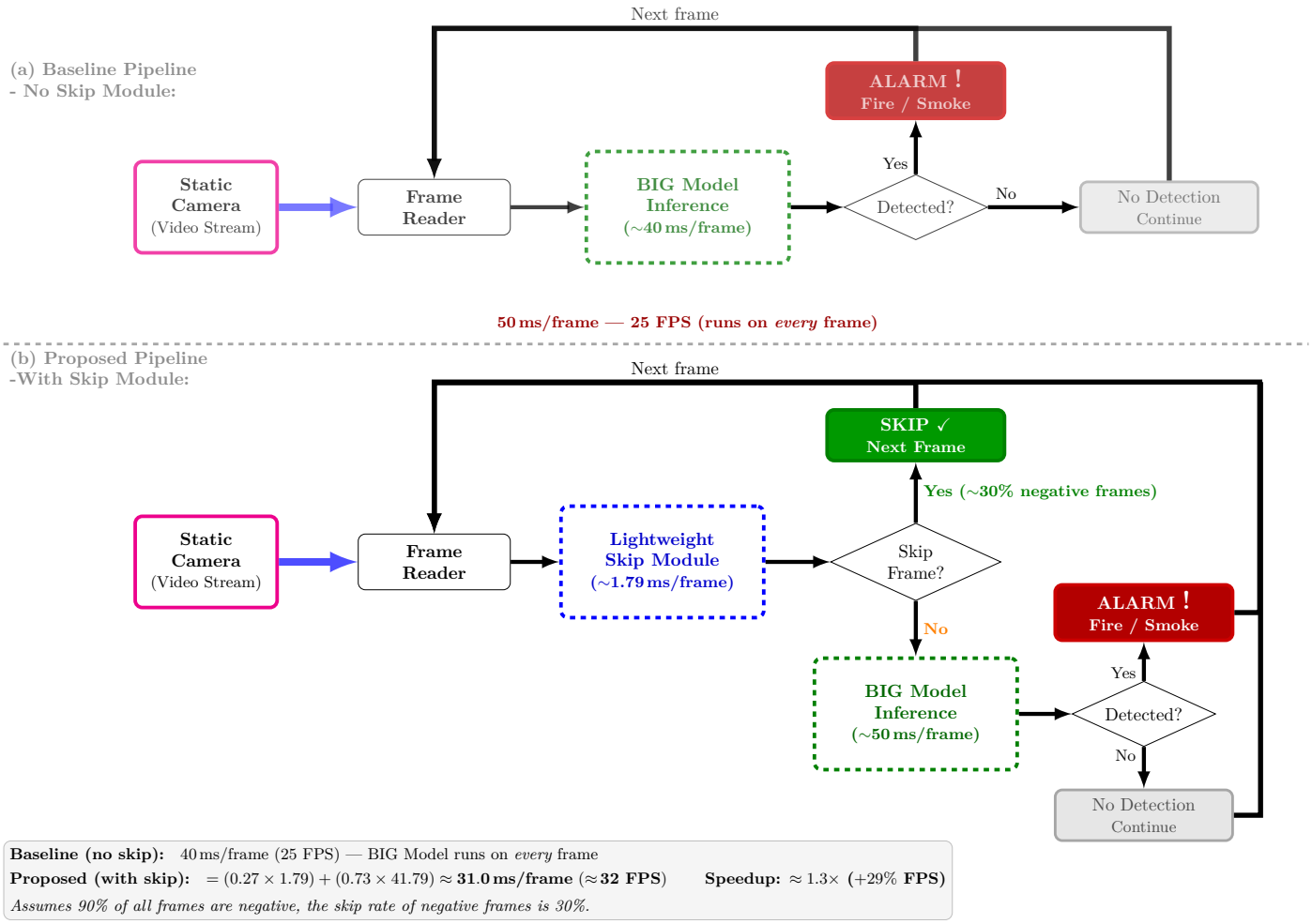


Figure 1: Conventional frame-wise inference pipeline (top) versus the proposed skip-module-enabled pipeline (bottom) for indoor fire and smoke detection.

Proposed system. The skip module $\mathcal{S}$ first evaluates f_t using inter-frame motion cues and outputs a binary gate decision s_t. While this work instantiates $\mathcal{S}$ using motion-based cues exclusively, the formulation is general and the skip module can incorporate other types of cues such as color, texture, or learned features. The full system output becomes:

$$\hat{y}_t = \begin{cases} \mathcal{M}(f_t) & \text{if } s_t = 1 \quad \text{(run inference)} \\ 0 & \text{if } s_t = 0 \quad \text{(skip, label as negative)} \end{cases}$$

In this work, skip module $\mathcal{S}$ derives s_t from the motion estimated between f_t and the previous frame f_{t-1}. Frames with little or no motion are unlikely to contain fire or smoke, and are therefore skipped without invoking the classifier $\mathcal{M}$. The model $\mathcal{M}$ is typically a high-capacity, accurate classifier but is computationally expensive. The skip module $\mathcal{S}$ is deliberately designed to be computationally lightweight, adding negligible overhead. The goal of the skip module $\mathcal{S}$ is to skip as many background/safe frames as possible while minimizing the risk of skipping fire/smoke frames, thus improving overall system throughput (FPS) while maintaining high detection accuracy.

3.2 Skip Module Design

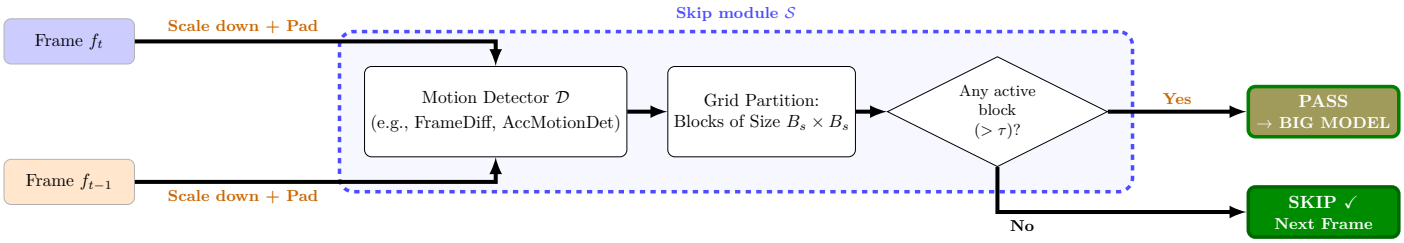


Figure 2: Skip Module for Fire and Smoke Detection

As illustrated in Figure 2 and formalized in Algorithm 1, the skip module $\mathcal{S}$ can leverage any available information — such as motion, color, or texture — to compute the skip decision s_t. In this work, we focus on motion information derived

Algorithm 1 Skip Module $\mathcal{S}$: Motion-Based Frame Gating

Require: Video stream $\{f_1, f_2, \dots\}$, motion detector $\mathcal{D}$, scale factor α, block size B, block active threshold τ
Ensure: Skip decisions $\{s_1, s_2, \dots\}$, where $s_t \in \{0, 1\}$

$\tilde{f}_{\text{prev}} \leftarrow$ null ▷ Initialize previous downsampled frame
for each incoming frame f_t **do**
  Step 1: Pre-process
  Downsample f_t by α to obtain $\tilde{f}_t$
  Let $B_s \leftarrow B \cdot \alpha$ ▷ Effective block size on downsampled frame
  Pad $\tilde{f}_t$ to be divisible by B_s
  Step 2: Compute Foreground Mask
  if $\tilde{f}_{\text{prev}} =$ null **then** ▷ No previous frame on first iteration
    $s_t \leftarrow 1$ ▷ Run inference on first frame
    $\tilde{f}_{\text{prev}} \leftarrow \tilde{f}_t$
    continue
  end if
  $\mathbf{M}_t \leftarrow \mathcal{D}(\tilde{f}_t, \tilde{f}_{\text{prev}})$ ▷ $\mathcal{D}$: e.g., FrameDiffDet or AccMotionDet
  Partition $\mathbf{M}_t$ into a grid of non-overlapping blocks of size B_s
  Mark a block as *active* if its foreground pixel ratio $> \tau$
  Step 3: Gating Decision
  if any active block exists **then**
    $s_t \leftarrow 1$ ▷ Motion detected, run DL inference
  else
    $s_t \leftarrow 0$ ▷ No motion, skip inference
  end if
  $\tilde{f}_{\text{prev}} \leftarrow \tilde{f}_t$ ▷ Update previous frame for next iteration
  yield s_t
end for

from consecutive frames as the primary cue. Specifically, we design and evaluate two motion-based approaches of $\mathcal{S}$: (1) FrameDiffDet — a frame differencing method, and (2) AccMotionDet — a motion accumulation method. Both approaches are selected for their simplicity and low computational cost, making them well-suited for real-time deployment. The details of each approach are described in the following subsections.

3.2.1 FrameDiffDet — Naive Motion Detection

Let F_t and F_{prev} denote the grayscale frames converted from the downsampled inputs $\tilde{f}_t$ and $\tilde{f}_{\text{prev}}$, respectively. The detector takes these two consecutive downsampled frames as input and produces a binary foreground mask $\mathbf{M}_t \in \{0, 255\}$ as output, where each pixel is marked active (255) if the absolute inter-frame intensity difference exceeds a sensitivity threshold τ_d, and inactive (0) otherwise. The procedure is formalized in Algorithm 2.

Algorithm 2 Frame Difference Detector (FrameDiffDet)

Require: Downsampled frames $\tilde{f}_t$, $\tilde{f}_{\text{prev}}$
Require: τ_d: pixel difference sensitivity threshold
Ensure: $\mathbf{M}_t \in \{0, 255\}$: binary foreground mask

Convert $\tilde{f}_t$ and $\tilde{f}_{\text{prev}}$ to grayscale: F_t, F_{prev}
$\mathbf{M}_t(i,j) \leftarrow \begin{cases} 255 & |F_t(i,j) - F_{\text{prev}}(i,j)| > \tau_d \\ 0 & \text{otherwise} \end{cases}$
return $\mathbf{M}_t$

3.2.2 AccMotionDet — Motion Detection with Accumulation

Let F_t and F_{prev} denote the grayscale frames converted from the downsampled inputs $\tilde{f}_t$ and $\tilde{f}_{\text{prev}}$, respectively, and let $\mathbf{K}_{t-1} \in [0, K_{\max}]$ denote the per-pixel *accumulated motion mask* from the previous frame. The detector takes two consecutive downsampled frames and $\mathbf{K}_{t-1}$ as input, and produces a binary foreground mask $\mathbf{M}_t \in \{0, 255\}$ and the updated mask $\mathbf{K}_t$ as output. $\mathbf{K}_t$ is incremented by ω when the inter-frame pixel difference exceeds τ_d, and decays by δ each frame, bounded at $K_{\max}$ to prevent unbounded growth under persistent motion. A pixel is marked active (255) only when $\mathbf{K}_t \geq \tau_m$, suppressing transient noise and single-frame flicker, and inactive (0) otherwise. The procedure is formalized in Algorithm 3.

3.3 Skip Module Hyperparameter Optimization

The skip module $\mathcal{S}$, as described in Algorithm 1, is governed by several hyperparameters — including scale factor α, block active threshold τ, and pixel difference sensitivity threshold τ_d — that can significantly affect overall system performance.

Algorithm 3 Accumulation Motion Detector (AccMotionDet)

Require: Downsampled frames $\tilde{f}_t$, $\tilde{f}_{\text{prev}}$; accumulated mask $\mathbf{K}_{t-1}$ (initialized to $\mathbf{0}$)
Require: τ_d: pixel difference sensitivity, ω: motion increment, τ_m: activation threshold, $K_{\max}$: accumulation cap, δ: decay rate
Ensure: $\mathbf{M}_t \in \{0, 255\}$: binary foreground mask; $\mathbf{K}_t$: updated accumulated mask

Convert $\tilde{f}_t$ and $\tilde{f}_{\text{prev}}$ to grayscale: F_t, F_{prev}

$\mathbf{D}_t \leftarrow |F_t - F_{\text{prev}}|$ ▷ Absolute inter-frame difference

$$\mathbf{\Delta}_t(i,j) \leftarrow \begin{cases} \omega & \mathbf{D}_t(i,j) > \tau_d \\ 0 & \text{otherwise} \end{cases}$$ ▷ Instantaneous increment map

$\mathbf{K}_t \leftarrow \min(\mathbf{K}_{t-1} + \mathbf{\Delta}_t,\ K_{\max})$ ▷ Accumulate and cap

$\mathbf{K}_t \leftarrow \max(\mathbf{K}_t - \delta,\ 0)$ ▷ Temporal decay

$$\mathbf{M}_t(i,j) \leftarrow \begin{cases} 255 & \mathbf{K}_t(i,j) \geq \tau_m \\ 0 & \text{otherwise} \end{cases}$$ ▷ Threshold to binary mask

return $\mathbf{M}_t$, $\mathbf{K}_t$

Algorithm 4 Skip Module Hyperparameter Selection

Require: Parameter space Θ, validation set D_{val}, DL model $\mathcal{M}$, recall tolerance δ_R, weights w_S, w_R
Ensure: Optimal parameters θ^*

baseline $\leftarrow$ EvaluatePipeline(D_{val}, skip=None, $\mathcal{M}$)
Extract R_{base} from baseline
$\Theta_{\text{feasible}} \leftarrow \emptyset$

▷ **Pass 1: Collect feasible candidates**

for each $\theta \in \Theta$ **do**
  results $\leftarrow$ EvaluatePipeline($D_{\text{val}}, \mathcal{S}(\theta), \mathcal{M}$)
  Extract $R(\theta)$, $S_r(\theta)$ from results
  if $R(\theta) \geq R_{\text{base}} - \delta_R$ **then**
    $\rho(\theta) \leftarrow 1 - \dfrac{R_{\text{base}} - R(\theta)}{\delta_R}$
    $\Theta_{\text{feasible}} \leftarrow \Theta_{\text{feasible}} \cup \{(\theta,\ \rho(\theta),\ S_r(\theta))\}$
  end if
end for

▷ **Pass 2: Range-normalize and select**

$$\tilde{\rho}(\theta) \leftarrow \frac{\rho(\theta) - \min_{\Theta_{\text{feasible}}} \rho}{\max_{\Theta_{\text{feasible}}} \rho - \min_{\Theta_{\text{feasible}}} \rho} \quad \text{for each } \theta \in \Theta_{\text{feasible}}$$

$$\tilde{S}_r(\theta) \leftarrow \frac{S_r(\theta) - \min_{\Theta_{\text{feasible}}} S_r}{\max_{\Theta_{\text{feasible}}} S_r - \min_{\Theta_{\text{feasible}}} S_r} \quad \text{for each } \theta \in \Theta_{\text{feasible}}$$

$\theta^* \leftarrow \arg\max_{\theta \in \Theta_{\text{feasible}}} [w_R\, \tilde{\rho}(\theta) + w_S\, \tilde{S}_r(\theta)]$
return θ^*

To identify the optimal configuration, we employ a constrained grid search over a validation set D_{val}. Let R_{base} denote the baseline recall without skipping. For each candidate $\theta \in \Theta$, we evaluate the full pipeline Read $\rightarrow \mathcal{S}(\theta) \rightarrow [\mathcal{M}]$ and compute recall $R(\theta)$ and skip rate $S_r(\theta)$ (see Section 4.2 for formal definitions).

Feasibility constraint. A candidate θ is feasible only if:

$$R(\theta) \geq R_{\text{base}} - \delta_R, \tag{1}$$

where $\delta_R > 0$ is the maximum allowable recall drop (e.g., $\delta_R = 0.01$ permits at most a 1% reduction relative to the baseline).

Scoring and selection. For each feasible candidate, we compute the recall retention score

$$\rho(\theta) = 1 - \frac{R_{\text{base}} - R(\theta)}{\delta_R}, \tag{2}$$

which maps the recall drop linearly onto $[0, 1]$ within the feasible region. Both $\rho(\theta)$ and $S_r(\theta)$ are range-normalized over Θ_{feasible} (Pass 2 of Algorithm 4), yielding $\tilde{\rho}(\theta)$ and $\tilde{S}_r(\theta) \in [0, 1]$, so that the weights w_S and w_R directly reflect their intended relative importance. The optimal configuration is then:

$$\theta^* = \arg \max_{\theta \in \Theta_{\text{feasible}}} \Big[w_R\, \tilde{\rho}(\theta) + w_S\, \tilde{S}_r(\theta) \Big], \tag{3}$$

where $\Theta_{\text{feasible}} = \{\theta \in \Theta : R(\theta) \geq R_{\text{base}} - \delta_R\}$ and $w_S + w_R = 1$.

FAR guarantee and exclusion from scoring. Let $FP(\theta)$ and FP_{base} denote the number of false positives produced by the skip-enabled and baseline systems, respectively, and let $N_{\text{skip\_neg}}(\theta)$ denote the number of ground-truth negative frames skipped by the skip module $\mathcal{S}$ under configuration θ (see Section 4.2 for definitions of FAR, S_r, and N_{neg}). Since every skipped frame is labeled negative, false positives arise only from frames passed to classifier $\mathcal{M}$, so $FP(\theta) = FP_{\text{base}} - a$ for some $a \geq 0$. Because a skipped false positive must simultaneously be a baseline false positive and a skipped negative frame, $a \leq \min(FP_{\text{base}}, N_{\text{skip\_neg}})$ without any assumption on the relationship between the skip module $\mathcal{S}$ and the classifier $\mathcal{M}$. Dividing by N_{neg} and substituting $\text{FAR}_{\text{base}} = FP_{\text{base}}/N_{\text{neg}}$ and $S_r(\theta) = N_{\text{skip\_neg}}/N_{\text{neg}}$ yields the assumption-free interval:

$$\text{FAR}(\theta) \in \left[\max(0,\ \text{FAR}_{\text{base}} - S_r(\theta)),\ \text{FAR}_{\text{base}}\right]. \tag{4}$$

The upper bound confirms that the skip module cannot increase the false alarm rate by design; the primary safety risk is therefore recall degradation caused by incorrectly skipped fire/smoke frames, which motivates enforcing the recall constraint as a hard feasibility gate. Explicit FAR optimization is nonetheless unnecessary: since $\mathcal{S}$ operates solely on inter-frame motion without access to $\mathcal{M}$'s outputs, its hyperparameters do not directly control which baseline false positives are removed, and including FAR in the objective could favor configurations that overfit to validation-specific error patterns. We accordingly report $\text{FAR}(\theta)$ only as an observed consequence metric in the results tables.

4 Experiments and Results

4.1 Fire and Smoke Indoor Video Dataset

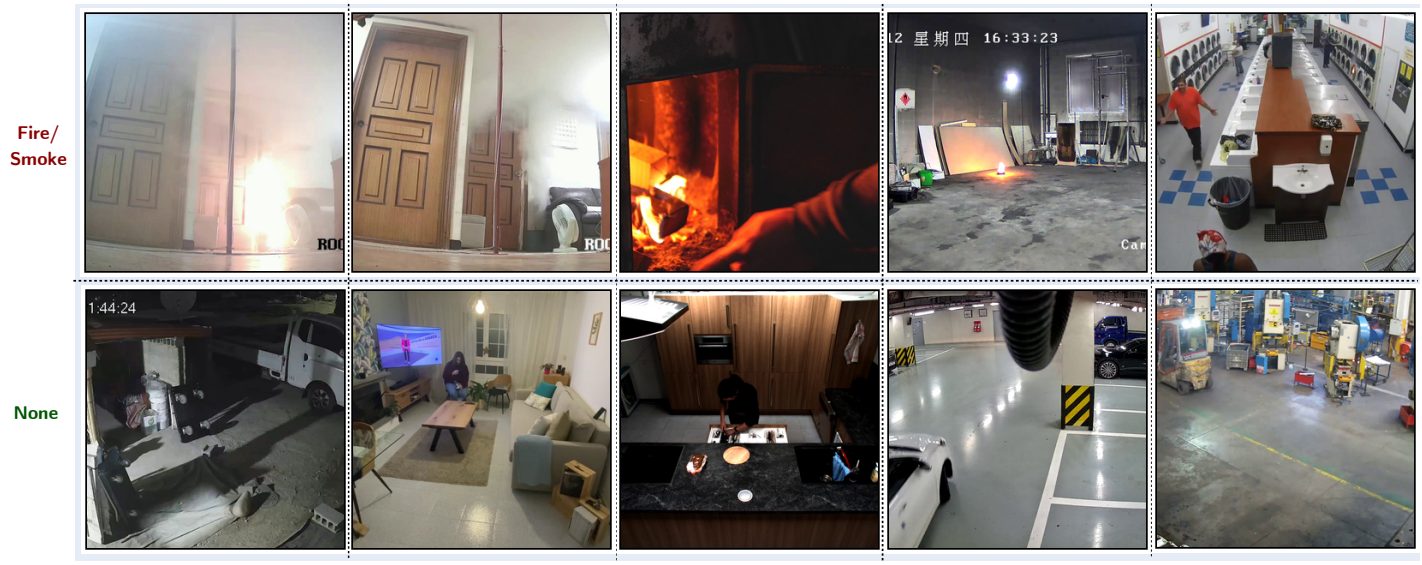


Figure 3: Representative frames extracted from the indoor fire and smoke video dataset used in this study.

Table 1: Comparison of existing fire and smoke video datasets with the proposed dataset (150 videos; static CCTV cameras; diverse indoor environments). Unlike most existing datasets, 143 out of 150 videos are recorded at HD resolution (1280×720) or above, matching commercial surveillance camera standards.

Model	Category	Videos	Resolution Min	Resolution Max	FireSmoke	Normal	Indoor	Outdoor
FireSense (2010) [17]	Moving camera	49	300 × 240	1600 × 1200	24	25	10	39
KMU (2011) [22]	Static camera	38	320 × 240	320 × 240	28	10	10	28
VisiFireBilkent (2015) [21]	Static camera	39	320 × 240	720 × 576	38	1	6	32
Mivia FIRE (2015) [23]	Static camera	31	320 × 240	800 × 600	28	3	4	27
Mivia SMOKE (2015) [23]	Static camera	149	292 × 240	292 × 240	76	73	0	149
FURG (2015) [19]	Moving camera	22	240 × 344	1920 × 1080	17	5	3	19
USTC Smoke (2017) [24]	Static camera	30	1920 × 1080	1920 × 1080	22	8	30	0
FireNet (2019) [16]	Moving camera	62	352 × 222	1920 × 1080	46	16	30	32
FiSmo (2020) [18]	Moving camera	88	320 × 240	1920 × 1080	80	8	0	88
D-Fire (2021) [20]	Static camera	100	1280 × 720	1280 × 720	50	50	0	100
VDS3 (2022) [26]	Mix camera	20	352 × 288	2048 × 1536	8	12	6	14
Ours (2026) - This work	Static camera	150	814 × 720	3840 × 2160	75	75	150	0

Several existing benchmarks address this detection task, including FireNet [16], Firesense [17], FiSmo [18], and FURG [19]. However, these were recorded with moving cameras and are therefore unsuitable for static surveillance scenarios. Static-camera datasets such as DFire [20], VisiFire Bilkent [21], KMU Fire and Smoke [22], Mivia Fire and Mivia Smoke datasets [23], and USTC Smoke [24] do exist; however, they collectively suffer from several limitations: a predominance of outdoor scenes, a small number of video samples, low spatial resolution, and insufficient diversity in fire/smoke appearances and environmental conditions.

To address these deficiencies, we constructed a dedicated static indoor video dataset by aggregating clips from multiple heterogeneous sources. Fire and smoke samples were sourced from the Korea AI Fire Dataset [25], the USTC Smoke Dataset [24], and the VDS3 Dataset [26], supplemented by videos collected from open online platforms (Pexels, Pixabay, and YouTube). Non-fire/smoke (negative) samples were compiled from self-recorded footage captured by real CCTV cameras in various indoor environments (e.g., parking areas), along with videos from the Safe & Unsafe Behavior in Workplaces dataset [27], the Indoor Action dataset [28], the MPII Cooking 2 Dataset [29], and the WiseNet dataset [30]. Table 1 summarizes the properties of the self-collected video dataset alongside a comparison with existing datasets, and Figure 3 presents representative sample frames from our dataset.

For the hyperparameter optimization described in Section 3.3, the video dataset was further partitioned into a validation set, D_{val} (46 videos), used to select the optimal skip-module hyperparameters, and a test set, D_{test} (104 videos), used for the final evaluation of the proposed system against the baseline and existing methods. The partition adhered to a 30:70 ratio (validation:test), following the protocol of [10], and was performed using stratified sampling to preserve balanced class distributions in both subsets.

4.2 Evaluation Metrics

Performance is evaluated under **frame-level** evaluation, following the standard protocol for fire and smoke detection in videos [31]. Frame-level evaluation measures detection performance for each individual frame, providing a strict assessment of classification accuracy. The following label definitions apply to this evaluation protocol:

- True Positive (TP): Correct detection of fire or smoke.
- True Negative (TN): Correct identification of the absence of fire or smoke.
- False Positive (FP): Incorrect detection of fire or smoke when none is present.
- False Negative (FN): Failure to detect fire or smoke when present.

Quantitative results are reported using standard classification metrics — accuracy, recall, false alarm rate (FAR), precision, F1-score, and frames per second (FPS) — as defined below.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{5}$$

$$\text{Recall (R)} = \frac{TP}{TP + FN}, \tag{6}$$

$$\text{False Alarm Rate (FAR)} = \frac{FP}{FP + TN}, \tag{7}$$

$$\text{Precision (P)} = \frac{TP}{TP + FP}, \tag{8}$$

$$\text{F1-score} = 2 \times \frac{P \times R}{P + R}, \tag{9}$$

$$\text{FPS} = \frac{\text{Total Frames}}{\text{Total Processing Time (seconds)}}, \tag{10}$$

$$\tag{11}$$

where TP, TN, FP, and FN represent True Positive, True Negative, False Positive, and False Negative, respectively.

In addition to standard metrics, we report the skip rate S_r, a system-level efficiency criterion specific to the proposed skip-based framework. Skip rate measures the proportion of ground-truth negative frames correctly bypassed by $\mathcal{S}$, defined as

$$S_r = \frac{N_{\text{skip\_neg}}}{N_{\text{neg}}} \tag{12}$$

where $N_{\text{skip\_neg}}$ is the number of ground-truth negative frames assigned $s_t = 0$ by the skip module $\mathcal{S}$ (i.e., background frames bypassed without invoking big DL model $\mathcal{M}$), and N_{neg} is the total number of ground-truth negative frames in the evaluated set. A higher S_r indicates greater computational savings, while a drop in recall may signals unsafe skipping of fire/smoke frames.

4.3 Models and Implementation Details

Baseline Models: To demonstrate both the superiority of the high-capacity classifier over lightweight alternatives and the effectiveness of the proposed skip module in accelerating its inference, we evaluate the following baseline models alongside our BIG model:

- **MobileNet** [32]: A modified MobileNet architecture fine-tuned for fire and smoke detection.
- **FireNet** [16]: A lightweight CNN-based image classifier comprising 14 layers, with a model size of 7.5 MB and approximately 650k trainable parameters.
- **YOLOv5s** [10]: A lightweight object detector based on YOLOv5 [33], trained with hyperparameters selected via grid search on the D-Fire dataset [20].
- **YOLOv5l** [10]: A heavier variant of the above, based on the larger YOLOv5l architecture, offering higher capacity at increased computational cost.

Proposed Classifier (BIG Model $\mathcal{M}$ — HGNetV2): The BIG model $\mathcal{M}$ is obtained by fine-tuning a pretrained High-Performance GPU Network v2 (HGNetV2), specifically the `hgnetv2_b5.ssld_stage2_ft_in1k` checkpoint [34] from the Timm library [35], which was originally trained using SSLD knowledge distillation [36]. HGNetV2 [37] is a high-capacity CNN architecture designed to achieve substantially higher accuracy than models of comparable inference speed on NVIDIA GPUs, making it well-suited for real-life deployment. For fine-tuning, a dataset of 253,325 fire/smoke/normal images was compiled from internet sources and partitioned into training (90%) and validation (10%) subsets. The model was trained for 100 epochs with a batch size of 64 using the Adam optimizer [38], configured with a learning rate of 0.003, weight decay of 0.05, and a warmup over the first 5 epochs. The training was conducted on a single NVIDIA RTX 5090 GPU (32 GB VRAM) paired with an Intel Core Ultra 9 285K processor and 128 GB of system RAM, running PyTorch 2.7.1 with CUDA Toolkit 12.8. All input images were preprocessed by resizing to $3 \times 360 \times 640 (C \times H \times W)$. The fine-tuned model achieved a classification accuracy of 98.34% and a high recall of 98.12% on the validation set.

Implementation Details: Unless stated otherwise, all experiments were conducted on a workstation equipped with an Intel Core i9-10900K CPU, 64 GB DDR4 system memory, and an NVIDIA GeForce RTX 3090 GPU (24 GB VRAM), running Windows 10 Pro (22H2, build 19045). Deep learning inference was performed using PyTorch 2.7.1 under CUDA 12.9, and video processing and motion estimation were carried out using OpenCV 4.11 (CPU-only).

4.4 Results

4.4.1 Hyperparameter Optimization Results

In this work, the recall tolerance is set to $\delta_R = 0.015$, permitting a maximum absolute recall drop of 1.5% relative to the baseline. At the baseline recall of approximately 95%, this ensures the worst-case deployed system retains a recall of at least 93.5%, while affording the hyperparameter search sufficient flexibility to identify configurations with meaningful efficiency gains. The optimization weights are set to $w_S = 0.7$ and $w_R = 0.3$, prioritizing skip rate (the primary determinant of throughput gain) while using recall retention as a secondary criterion to favor configurations closer to baseline performance among feasible candidates.

4.4.1.1 Hyperparameter Optimization Results for the FrameDiffDet-based Skip Module To identify the optimal configuration for the FrameDiffDet-based skip module, we performed a systematic grid search over four hyperparameters: scale factor α, block size B, block active threshold τ, and difference sensitivity threshold τ_d, yielding a total of $2 \times 2 \times 3 \times 4 = 48$ candidate configurations. The search space is as follows:

- **Scale factor** $\alpha \in \{0.5, 1.0\}$: controls the downsampling ratio applied to input frames prior to motion computation. A smaller α reduces computational cost but may lose subtle motion detail. We evaluate full resolution ($\alpha = 1.0$) and half resolution ($\alpha = 0.5$).
- **Block size** $B \in \{16, 32\}$: determines the spatial granularity of motion detection, expressed in pixels of the original frame (the effective block size on the downsampled frame is $B_s = B \cdot \alpha$). Smaller blocks ($B = 16$) capture localized motion from small fire or smoke regions, while larger blocks ($B = 32$) aggregate motion over a broader spatial context, providing greater robustness against isolated pixel-level noise.
- **Block active threshold** $\tau \in \{0.05, 0.10, 0.15\}$: defines the minimum foreground pixel ratio within a block for it to be declared active; inference is triggered if at least one active block is detected. A low value ($\tau = 0.05$) is sensitive to sparse motion within a block, minimizing missed detections, while a higher value ($\tau = 0.15$) demands denser local motion, suppressing responses to minor pixel-level disturbances. The three values provide a systematic sweep from fine to coarse sensitivity.
- **Difference sensitivity threshold** $\tau_d \in \{3, 5, 7, 10\}$: defines the minimum absolute inter-frame intensity change required for a pixel to be declared active. This range is chosen to cover diverse sensitivity levels, from near-noise-level detection ($\tau_d = 3$, responding to subtle illumination changes) to robust large-motion detection ($\tau_d = 10$, requiring strong, unambiguous motion).

Table 2 presents representative configurations illustrating the recall–efficiency trade-off; the full search space of 48 configurations is omitted for brevity. The results reveal a fundamental limitation of FrameDiffDet: it fails to deliver meaningful throughput improvement under the safety constraint $\delta_R = 0.015$.

Among all 48 evaluated configurations, only three satisfy the recall constraint, all requiring the lowest sensitivity setting $\tau_d = 3$. The best-ranked feasible configuration (α=0.5, B=16, τ=0.05, τ_d=3) achieves a skip rate of merely 0.94%, bypassing fewer than 1 in 100 background frames. Consequently, the FPS of all feasible configurations (22.0–23.3) falls below the no-skip baseline (24.7), indicating that the skip module overhead outweighs the benefit of skipped inference calls. Conversely,

Table 2: Representative hyperparameter configurations for FrameDiffDet on the validation set, illustrating the recall–efficiency trade-off across the full search space of 48 configurations (not all shown for brevity). Configurations with a ranking score of "-" violate the recall constraint ($\delta_R = 0.015$) and are included solely to illustrate the trade-off at higher skip rates. Recall (%) reports the retained recall with the absolute drop relative to the baseline in parentheses, i.e., R ($-\Delta R$). The best and second-best values per metric are highlighted in **bold** and *italic underline*, respectively.

Exp.	Parameters				Metrics				Ranking Score
	Scale factor α	Block size B	Block active threshold τ	Diff threshold τ_d	Recall (%) ↑	Skip Rate (%) ↑	FAR (%) ↓	FPS ↑	
Baseline (No Skip module)	-	-	-	-	**94.35** (-0.00)	0.0	0.74	24.7	-
FrameDiff	0.5	16.0	0.05	3.0	93.97 (-0.38)	0.94	0.737	23.3	**0.8046**
FrameDiff	1.0	16.0	0.1	3.0	93.54 (-0.81)	0.97	0.737	22.3	*0.7000*
FrameDiff	1.0	16.0	0.05	3.0	*94.18* (-0.17)	0.72	0.737	22.0	0.3000
FrameDiff	0.5	16.0	0.1	3.0	92.83 (-1.51)	1.35	0.737	23.6	-
FrameDiff	1.0	16.0	0.15	3.0	92.05 (-2.30)	1.53	0.737	22.5	-
FrameDiff	1.0	32.0	0.05	3.0	91.63 (-2.72)	2.43	0.737	23.2	-
FrameDiff	1.0	16.0	0.05	5.0	91.46 (-2.88)	1.94	0.737	22.6	-
FrameDiff	0.5	16.0	0.15	3.0	91.45 (-2.90)	3.92	0.737	24.3	-
FrameDiff	0.5	32.0	0.05	3.0	90.98 (-3.36)	7.11	0.737	25.3	-
FrameDiff	0.5	16.0	0.05	5.0	90.55 (-3.80)	6.23	0.737	24.7	-
FrameDiff	0.5	16.0	0.1	7.0	88.87 (-5.47)	47.22	*0.733*	37.3	-
FrameDiff	0.5	16.0	0.05	10.0	88.19 (-6.16)	*49.0*	*0.733*	*38.3*	-
FrameDiff	0.5	32.0	0.05	10.0	84.56 (-9.79)	**59.53**	**0.73**	**45.5**	-

configurations that do achieve substantial skip rates (reaching 47–60% at higher τ_d values of 7 and 10) suffer severe recall degradation of 5–10%, far exceeding the safety tolerance.

These results demonstrate that FrameDiffDet, without noise suppression or robust sustained-motion confirmation, is unsuitable as a skip gate: it either skips too few frames to yield any efficiency gain, or skips too aggressively and suffer unacceptable recall loss.

4.4.1.2 Hyperparameter Optimization Results for the AccMotionDet-based Skip Module

The hyperparameter search for AccMotionDet is designed to mirror the FrameDiffDet study while accounting for the temporal accumulation of motion scores over consecutive frames. Details of the search space (of 96 configurations) are as follows:

- **Scale factor** ($\alpha \in \{0.5, 1.0\}$) and **Block size** ($B \in \{16, 32\}$): Retained from FrameDiffDet to maintain consistency in the spatial resolution and grid density trade-offs.
- **Block active threshold** ($\tau \in \{0.05, 0.10\}$): Restricted to a narrower range than in FrameDiffDet, as temporal accumulation naturally suppresses false activations, rendering more aggressive thresholds unnecessary.
- **Diff threshold** ($\tau_d \in \{3, 5\}$): Limited to the lower end of the FrameDiffDet search space, as higher values were found to cause excessive frame skipping and unacceptable recall degradation.
- **Accumulation step** ($\omega = 5$) and **Decay rate** ($\delta = 1$): We fix these parameters to streamline the hyperparameter search, allowing us to focus on the primary sensitivity controls: the activation threshold (τ_m) and accumulation cap ($K_{\max}$). Since the optimal tuning of τ_m and $K_{\max}$ depends directly on by ω, keeping ω constant provides a stable baseline for optimization. Furthermore, setting $\delta = 1$ ensures consistent, linear decay behavior while reducing the dimensionality of the search space.
- **Activation threshold** ($\tau_m \in \{5, 10\}$): This threshold serves as the decision gate for the motion mask, labeling pixels as active only when their accumulated motion score exceeds τ_m. Given the fixed accumulation parameters ($\omega = 5, \delta = 1$), $\tau_m = 5$ and $\tau_m = 10$ are approximately equivalent to requiring motion signals over 2 and 3 consecutive frames, respectively.
- **Accumulation cap** ($K_{\max} \in \{15, 25, 35\}$): This parameter limits the maximum accumulated motion score to prevent runaway accumulation during prolonged activity. Without this cap, the system could remain triggered indefinitely even after motion ceases. By selecting a range of values, we diversify the system' s saturation behavior, allowing us to tune how quickly the module resets following high-activity periods.

Table 3 presents representative configurations illustrating the recall–efficiency trade-off for AccMotionDet; the full search space is omitted for brevity. These results demonstrate a substantial improvement over the naive over the naive FrameDiffDet skip module: by leveraging temporal accumulation, AccMotionDet maintains high recall while consistently achiev-

Table 3: Validation-set performance and hyperparameter configurations for Accumulation Motion Detector (AccMotionDet) combinations. Note that the "Recall (%)" column displays the retained recall alongside the absolute recall drop relative to the baseline in parentheses, i.e., Recall (-Recall Drop). The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively. Note that Motion increment $\omega = 5$ and Decay rate $\delta = 0.1$ are fixed across all configurations.

Method	Parameters										Ranking Score
	Scale factor α	Block size B	Block active threshold τ	Diff threshold τ_d	Activation threshold τ_m	Acc. cap $K_{\max}$	Recall (%)↑	FPR (%)↓	Skip Rate (%)↑	FPS ↑	
Baseline (No Skip module)	-	-	-	-	-	-	**94.35** (-0.00)	0.74	0.0	24.7	-
AccMotionDet	0.5	32.0	0.05	5.0	5.0	15.0	93.9 (-0.45)	0.737	44.39	36.3	**0.8020**
AccMotionDet	0.5	32.0	0.1	5.0	5.0	25.0	93.33 (-1.02)	0.737	55.09	41.1	0.7912
AccMotionDet	0.5	32.0	0.05	5.0	5.0	25.0	93.92 (-0.42)	0.737	42.89	35.7	0.7903
AccMotionDet	0.5	32.0	0.1	5.0	5.0	35.0	93.35 (-0.99)	0.737	54.42	40.8	0.7898
AccMotionDet	0.5	32.0	0.05	5.0	5.0	35.0	93.93 (-0.42)	0.737	42.38	35.5	0.7856
AccMotionDet	0.5	32.0	0.1	5.0	5.0	15.0	93.22 (-1.13)	0.737	**56.3**	**41.8**	0.7785
AccMotionDet	0.5	32.0	0.1	3.0	5.0	15.0	94.02 (-0.33)	0.737	39.76	34.5	0.7756
AccMotionDet	0.5	32.0	0.1	3.0	5.0	25.0	94.02 (-0.33)	0.737	39.36	34.3	0.7705
AccMotionDet	0.5	32.0	0.1	3.0	5.0	35.0	94.02 (-0.33)	0.737	39.09	34.2	0.7671
AccMotionDet	0.5	32.0	0.05	3.0	10.0	15.0	93.81 (-0.53)	**0.735**	42.63	35.7	0.7591

ing skip rates between 39% and 56%. The best-ranked configuration (α=0.5, B=32, τ=0.05, τ_d=5, τ_m=5, $K_{\max}$=15) achieves a skip rate of 44.39% with only a 0.45% drop in recall, yielding a throughput of 36.3 FPS—a significant improvement over the 24.7 FPS baseline. In contrast to earlier configurations that failed to bridge the throughput gap, the AccMotionDet configurations successfully operate within the safety constraint $\delta_R = 0.015$, confirming that temporal accumulation is requisite for robust motion-based gating.

4.4.2 Frame-Level Performance and Throughput: Baselines and Skip Module

Table 4 benchmarks the proposed pipeline against lightweight fire-and-smoke detectors, described in Section 4.3. The baseline BIG Model (without skip module) achieves the highest recall (95.62%), accuracy (98.77%), and F1-score (0.97), substantially outperforming all lightweight alternatives. This gap reflects the consequences of neural scaling laws [39], [40], [41]: the BIG model benefits from both a more expressive architecture and a significantly larger training dataset. Notably, MobileNet and FireNet exhibit near-degenerate recall of 5.84% and 34.1%, respectively, rendering them unsuitable for safety-critical fire and smoke detection despite their speed and compactness. YOLOv5s achieves the highest throughput (109.94 FPS) but still suffers a recall of 68.58%, roughly 27 percentage points below the BIG model, confirming that no lightweight alternative achieves an acceptable accuracy–efficiency balance on this task.

Effect of the Skip Module: Integrating the skip module yields a consistent set of improvements across nearly all metrics. Throughput increases from 24.97 to 32.32 FPS, a gain of approximately 29%, while the F1-score is fully preserved at 0.97. Recall drops by only 1.24 percentage points (95.62% → 94.38%), remaining within the safety constraint $\delta_R = 0.015$. A less obvious but notable benefit is the reduction in FAR from 0.25% to 0.14%, accompanied by a precision increase from 99.17% to 99.54%: by suppressing inference on near-static background frames, the skip module also eliminates a class of spurious detections that the BIG model would otherwise generate.

4.4.3 Comparison with Temporal Post-Processing

Temporal Persistence Thresholding (TPT) [10] augments the standard per-frame inference pipeline with a lightweight post-processing stage designed to suppress isolated false alarms. Unlike the proposed skip module, TPT does not reduce the number of frames forwarded to the classifier — every frame is still processed by the full BIG model. Instead, a circular boolean buffer of length W records the raw per-frame predictions over a rolling window. A detection is confirmed only if the fraction of positive predictions within the buffer exceeds a persistence threshold τ_{persist}; otherwise, the prediction is suppressed and the frame is re-labelled as negative.

We follow the hyperparameter selection protocol of [10], evaluating TPT on the validation set $\mathcal{D}_{\text{val}}$. Two representative configurations are reported: a strict variant ($\text{TPT}_{\text{strict}}$: $W = 5$, $\tau_{\text{persist}} = 0.2$) and a balanced variant ($\text{TPT}_{\text{balanced}}$: $W =$

Table 4: Frame-level accuracy and throughput of the proposed pipeline against lightweight fire-and-smoke detectors on the test set. "Our Big Model (with Skip Module)" refers to the full proposed pipeline. Recall, FAR, Accuracy, and Precision are reported in percentage (%); FPS is averaged across all frames in the test videos. GFLOPs denotes the static per-frame floating-point operations (10^9) of the detection model. The best and second-best values per metric are highlighted in **bold** and *italic underline*, respectively.

Method	Recall ↑	FAR ↓	Accuracy ↑	F1-Score ↑	Precision ↑	FPS ↑	GFLOPs ↓	Model size (Mb) ↓
MobileNet [32]	5.84	0.18	77.49	0.11	91.06	35.15	1.135	20.6
FireNet [16]	34.1	13.47	74.07	0.38	44.11	48.12	**0.018**	**7.45**
YOLOv5l [10]	49.36	3.61	85.22	0.61	81.0	78.08	107.7	88.5
YOLOv5s [10]	68.58	10.39	84.61	0.68	67.29	**109.94**	15.8	13.7
Our Big Model (without Skip Module)	**95.62**	0.25	**98.77**	**0.97**	99.17	24.97	30.61	430.0
Our Big Model (with Skip Module)	94.38	**0.14**	98.56	**0.97**	**99.54**	32.32	30.61	430.0

Table 5: Hyperparameter grid search for Temporal Persistence Thresholding (TPT) on the validation set. The best and second-best values per metric are highlighted in **bold** and *italic underline*, respectively.

Experiment	Window Size W	Persist Threshold τ_p	Recall (%) ↑	FAR (%) ↓	Accuracy (%) ↑	Precision (%) ↑	F1 Score ↑	FPS ↑
Big Model without Skip Module (Baseline)	-	-	**94.35** (-0.00)	0.7403	**98.26**	97.02	**0.9566**	**24.6617**
TPT method (Conservative: safety-first)	5.0	0.2	94.22 (-0.13)	0.7385 (-0.0017)	98.24	97.02	0.956	24.6546
TPT method (Balanced between Recall Drop and FAR decrease gain)	10.0	0.5	93.7 (-0.65)	**0.7315** (-0.0087)	98.13	**97.03**	0.9534	24.6544

Table 6: Frame-level performance comparison between the no-skip baseline, TPT variants, and the proposed pipeline (Big Model with AccMotionDet skip module) on the test set. The best and second-best values per metric are highlighted in **bold** and *italic underline*, respectively.

Method	Recall (%) ↑	FAR (%) ↓	Accuracy (%) ↑	Precision (%) ↑	F1 Score ↑	Skip Rate (%) ↑	FPS ↑
Baseline (notemp)	**95.616**	0.251	**98.767**	99.166	**0.9736**	0.0	24.97
$\text{TPT}_{\text{strict}}(W = 5,\ \tau_{\text{persist}} = 0.2)$	95.519	0.25	98.745	99.168	0.9731	0.0	24.96
$\text{TPT}_{\text{balanced}}(W = 10,\ \tau_{\text{persist}} = 0.5)$	95.123	0.247	98.653	99.174	0.9711	0.0	24.96
Big Model with Skip module (AccMotionDet)	94.384	**0.136**	98.562	**99.541**	0.9689	**34.93**	32.32

10, $\tau_{\text{persist}} = 0.5$), with the full grid search results shown in Table 5. Both configurations are then compared against the proposed pipeline in Table 8.

Table 8 highlights a clear distinction between temporal post-processing and the proposed skip module in terms of the efficiency–accuracy trade-off. TPT processes every frame through the full Big Model and therefore does not improve throughput, with FPS remaining essentially unchanged (24.97 to 24.96). In contrast, the AccMotionDet-based skip module skips 34.93% of negative frames and increases FPS from 24.97 to 32.32, corresponding to an approximately 29% throughput gain. It also achieves the largest reduction in FAR, decreasing from 0.251% to 0.136% (about 46% relative reduction), whereas the TPT variants provide only negligible FAR improvement. Among the two TPT settings, $\text{TPT}_{\text{balanced}}$ attains slightly lower recall than $\text{TPT}_{\text{strict}}$ (95.123% vs. 95.519%) while producing nearly the same FAR (0.247% vs. 0.250%), indicating limited sensitivity to the persistence threshold. Finally, the recall reduction of AccMotionDet is modest (95.62% → 94.38%) and remains within the safety constraint $\delta_R = 0.015$.

4.4.4 Quantitative and Qualitative Analysis

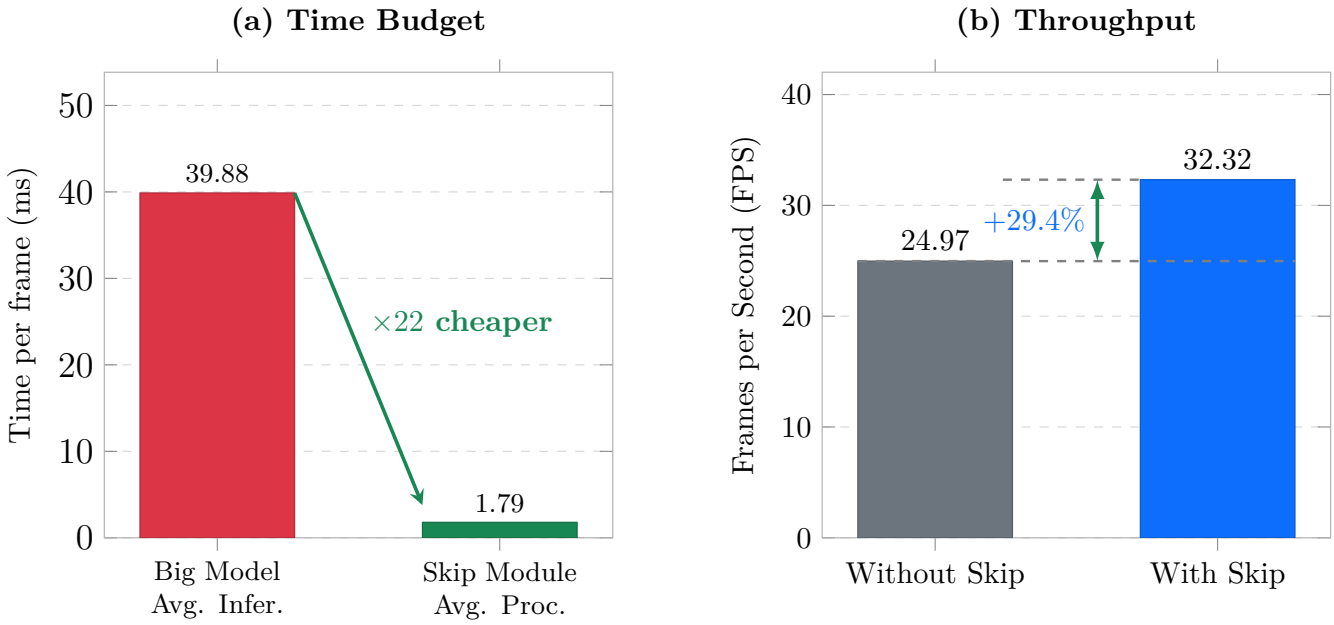


Figure 4: Computational efficiency of the proposed skip-module: (a) average processing time of the skip-module (1.79 ms) compared to the BIG model inference time (39.88 ms), representing a ×22 reduction; (b) system throughput with and without the skip-module, yielding a 29.4% improvement in FPS.

4.4.4.1 Efficiency of Skip Module To evaluate the efficiency of the skip module, the BIG model was run on the test set $\mathcal{D}_{\text{test}}$ with and without the skip module. The average processing time of the skip module, the BIG model inference time, and the overall system FPS were recorded in both cases. The results are visualized in Figure 4.

As shown in Figure 4, the skip module requires an average processing time of only 1.79 ms per frame, which is approximately ×22 lower than the BIG model inference time of 39.88 ms. In the best case, the skip module correctly identifies and bypasses negative frames, contributing only 1.79 ms of overhead. In the worst case, where the skip module fails to bypass a frame and the BIG model inference is subsequently invoked, the additional latency introduced by the skip module is 1.79 ms — a mere 4.5% increase relative to the BIG model inference time. Overall, integrating the skip module improves system throughput by approximately 30% on $\mathcal{D}_{\text{test}}$, demonstrating its effectiveness for real-time fire and smoke detection in indoor surveillance scenarios.

4.4.4.2 Qualitative Analysis To understand how the skip module operates in coordination with the Big Model, we visualize the inference results of the AccMotionDet skip module on the test set $\mathcal{D}_{\text{test}}$ in RGB images together with the corresponding foreground motion masks generated by the skip module. The qualitative examples are organized into two major categories: success cases and failure cases.

Success Cases

Figure 5 shows four representative success cases in which the skip module $\mathcal{S}$ operates as intended. These cases can be grouped into two categories:

- **Correctly skipped static background frames** (Fig. 5a,b): AccMotionDet identifies static background frames and skips them without invoking the Big Model. The corresponding motion masks are empty, indicating no active motion regions.

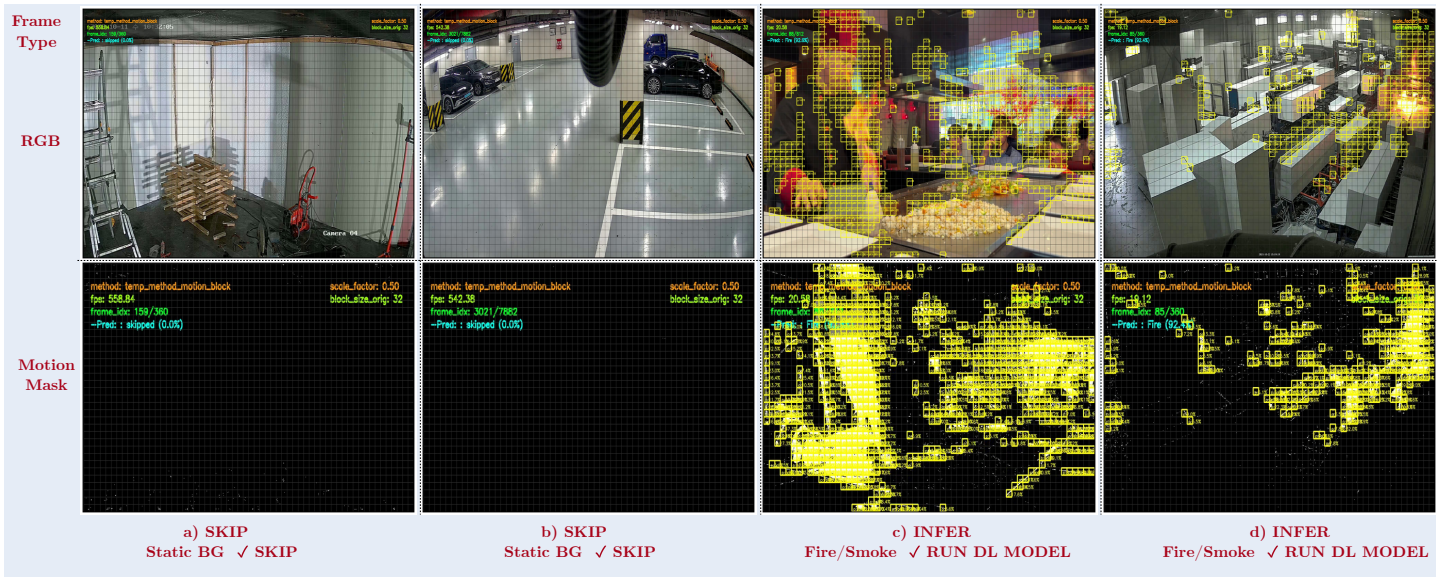


Figure 5: Qualitative results illustrating successful skip module operations. (a) and (b) Static background frames are correctly identified and skipped, avoiding unnecessary inference. (c) and (d) Frames containing active fire or smoke trigger the motion accumulator, correctly prompting the full deep learning model to evaluate the frame.

- **Correctly inferred fire/smoke frames** (Fig. 5c,d): The module correctly detects active fire and smoke regions and forwards these frames for inference. The yellow boxes indicate motion-active blocks detected by AccMotionDet, which are concentrated in the fire/smoke regions, consistent with the continuous nature of these phenomena.

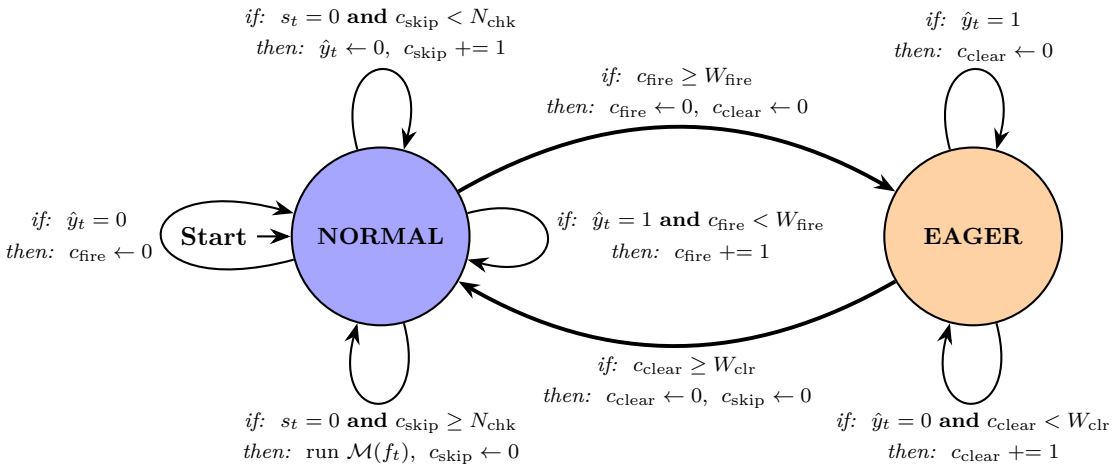
Failure Cases

Figure 6: Qualitative analysis of skip module failure modes. (a) Missed detection: slow-onset smoke fails to trigger the motion accumulator. (b–c) Wasted inferences: inaccurate motion detection OR persistent motion (e.g., human activity) saturates the accumulator, preventing frame skipping. (d–e) Big Model error: frames are correctly passed to the deep learning model, yet the model produces incorrect predictions.

Figure 6 shows five representative failure cases, which can be grouped into three categories:

- **Wrongly skipped frames** (Fig. 6a): Slowly developing smoke or smoke in its settling phase produces motion that is too subtle to trigger the skip module, leading to missed detections.
- **Wrongly passed frames / wasted inference** (Fig. 6b,c): Noise or persistent non-fire motion prevents background frames from being skipped, resulting in unnecessary inference.
- **Incorrect Big Model predictions** (Fig. 6d,e): Even when fire or smoke frames are correctly forwarded, the Big Model may still produce an incorrect label, especially when the target occupies only a small portion of the frame.

4.4.4.3 Eager Mode: Recall–FAR Trade-off A key failure mode of the skip module, illustrated in Figure 6a, is the missed detection of slowly developing or settling smoke, whose subtle inter-frame motion falls below the activation threshold of AccMotionDet. To address this, we augment the skip module with an *eager mode*: once the Big Model $\mathcal{M}$ produces a fire or smoke prediction on K_{confirm} consecutive non-skipped frames, the module enters eager mode and suppresses all skip decisions until the scene is declared safe again (i.e., $\mathcal{M}$ returns a negative prediction for W_{clr} consecutive frames). This



Symbol	Description
NORMAL	Skip module $\mathcal{S}$ is **active**: static frames are skipped; $\mathcal{M}$ is invoked only when motion is detected or $c_{\text{skip}} \geq N_{\text{chk}}$
EAGER	Skip module $\mathcal{S}$ is **bypassed**: $\mathcal{M}$ runs on *every* frame to prevent missed detections
$s_t \in \{0, 1\}$	Skip decision from $\mathcal{S}$: s_t=0 no motion detected (skip inference); s_t=1 motion detected (run inference)
$\hat{y}_t \in \{0, 1\}$	Prediction from $\mathcal{M}$: $\hat{y}_t$=1 fire/smoke; $\hat{y}_t$=0 negative
c_{skip}	Consecutive skipped frames (reset to 0 on any inference call)
c_{fire}	Consecutive fire/smoke predictions in NORMAL mode (reset to 0 on any negative)
c_{clear}	Consecutive negative predictions in EAGER mode
W_{fire}	Fire streak length required to enter EAGER mode
W_{clr}	Clear streak length required to exit EAGER mode
N_{chk}	Maximum consecutive skips before a forced inference check

Figure 7: State machine of the eager mode extension to the AccMotionDet-based skip module. Each transition is annotated with an *if* condition and a *then* action specifying variable updates. In NORMAL mode, $\mathcal{S}$ gates inference: frames with no detected motion are skipped and c_{skip} is incremented until $c_{\text{skip}} \geq N_{\text{chk}}$, at which point a forced inference check is issued and c_{skip} is reset. When $\mathcal{M}$ produces W_{fire} consecutive fire or smoke predictions ($c_{\text{fire}} \geq W_{\text{fire}}$), the system transitions to EAGER mode, where $\mathcal{S}$ is bypassed and every frame is forwarded to $\mathcal{M}$ to prevent missed detections during slow-developing or settling smoke events. The system returns to NORMAL mode only after W_{clr} consecutive negative predictions confirm that the scene is safe.

mechanism ensures that frames immediately following a confirmed detection are always forwarded to $\mathcal{M}$, recovering recall on slow-developing smoke events.

Figure 7 shows the state machine of eager mode with the transition conditions and corresponding actions for each state and how states transition during system operation.

To find the optimal hyperparameters for eager mode, we perform a grid search over a list of candidate values as follows:

- **Confirmation window** ($K_{\text{confirm}} \in \{3, 5\}$): This parameter defines the number of consecutive positive predictions required to trigger eager mode. A smaller value (e.g., $K_{\text{confirm}} = 3$) allows for quicker activation of eager mode, potentially recovering more slow-developing smoke cases, but may also increase false alarms. A larger value (e.g., $K_{\text{confirm}} = 5$) provides a more conservative trigger, reducing false alarms at the risk of missing some slow-onset events.
- **Clearance window** ($W_{\text{clr}} \in \{5, 10\}$): This parameter defines the number of consecutive negative predictions required to exit eager mode and resume normal skip operation. A shorter clearance window (e.g., $W_{\text{clr}} = 5$) allows the system to return to normal operation more quickly after a confirmed detection, but may also lead to premature exit from eager mode if there are intermittent false negatives. A longer clearance window (e.g., $W_{\text{clr}} = 10$) provides a more robust exit condition, ensuring that the system remains in eager mode until the scene is consistently declared safe, but may also delay the return to normal operation.
- **Period check interval** ($n_{\text{check}} \in \{10, 20\}$): This parameter defines how frequently the system checks for the conditions to enter or exit eager mode. A shorter interval (e.g., $n_{\text{check}} = 10$) allows for more responsive transitions between modes, but may also increase computational overhead. A longer interval (e.g., $n_{\text{check}} = 20$) reduces the frequency of checks, lowering overhead but potentially delaying mode transitions.

We also follow the same hyperparameter selection protocol as described in Section 3.3 to selected the optimal configuration

Table 7: Validation-set performance and hyperparameter configurations for Accumulation Motion Detector in Eager Mode (AccMotionDet Eager). Note that the "Recall (%)" column displays the retained recall alongside the absolute recall drop relative to the baseline in parentheses, i.e., Recall (-Recall Drop). The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively.

Method	Parameters		Metrics				Ranking Score
	N_{chk}	W_{clr}	Recall (%)↑	FPR (%)↓	Skip Rate (%)↑	FPS ↑	
Baseline (No Skip module)	-	-	**94.35** (-0.00)	0.74	0.0	24.1	-
AccMotionDet	-	-	93.9 (-0.00)	**0.737**	**44.39**	36.6	-
AccMotionDet (Eager Mode)	50	3	**94.35** (-0.00)	0.74	43.78	**36.7**	**0.7000**
AccMotionDet (Eager Mode)	50	5	**94.35** (-0.00)	0.74	43.75	**36.7**	0.6931
AccMotionDet (Eager Mode)	50	7	**94.35** (-0.00)	0.74	43.72	**36.7**	0.6866
AccMotionDet (Eager Mode)	50	10	**94.35** (-0.00)	0.74	43.66	36.6	0.6746
AccMotionDet (Eager Mode)	30	3	**94.35** (-0.00)	0.74	43.38	36.5	0.6128
AccMotionDet (Eager Mode)	30	5	**94.35** (-0.00)	0.74	43.35	36.5	0.6059
AccMotionDet (Eager Mode)	30	7	**94.35** (-0.00)	0.74	43.33	36.5	0.6005
AccMotionDet (Eager Mode)	30	10	**94.35** (-0.00)	0.74	43.26	36.5	0.5863
AccMotionDet (Eager Mode)	20	3	**94.35** (-0.00)	0.74	42.53	36.1	0.4261
AccMotionDet (Eager Mode)	20	5	**94.35** (-0.00)	0.74	42.5	36.1	0.4195
AccMotionDet (Eager Mode)	20	7	**94.35** (-0.00)	0.74	42.47	36.1	0.4134

on the validation set $\mathcal{D}_{\text{val}}$, and the results are shown in Table **??**.

The best-ranked configuration ($K_{\text{confirm}} = 3$, $W_{\text{clr}} = 10$, $n_{\text{check}} = 10$) then evaluated on the test set $\mathcal{D}_{\text{test}}$ and compared against the skip-only configuration in Table **??**.

Table~**??** reports the effect of eager mode. Recall is recovered from 94.38% (skip only) to 95.51%, matching the baseline within 0.11 percentage points, while the skip rate is marginally reduced from 34.93% to 34.21%. However, eager mode also partially negates the FAR improvement: FAR rises from 0.136% (skip only) back toward the baseline value of 0.251%. This outcome reflects a fundamental structural trade-off: eager mode cannot distinguish static false-alarm-prone scenes — which the Big Model persistently misclassifies — from genuine slow-smoke events, as both exhibit near-zero inter-frame motion. Consequently, the two operating modes of the skip module serve complementary deployment objectives: the *skip-only* configuration prioritises false alarm reduction (FAR ↓ 46%, FPS ↑ 29%), while the *skip + eager* configuration prioritises recall preservation at full baseline performance (recall ≥ 95.5%, FPS ↑ 29%).

Table 8: Comparison of Video-level performance metrics, measuring the delay and initial detection speed. The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively.

Method	Recall (%) ↑	FAR (%) ↓	Accuracy (%) ↑	Precision (%) ↑	F1 Score ↑	Skip Rate (%) ↑	FPS ↑
Baseline (notemp)	**95.616**	0.251	**98.767**	99.166	**0.9736**	0.0	24.97
$\text{TPT}_{\text{strict}}(W = 5,\ \tau_{\text{persist}} = 0.2)$	95.519	0.25	98.745	99.168	0.9731	0.0	24.96
$\text{TPT}_{\text{balanced}}(W = 10,\ \tau_{\text{persist}} = 0.5)$	95.123	0.247	98.653	99.174	0.9711	0.0	24.96
Big Model with Skip module (AccMotionDet)	94.384	**0.136**	98.562	**99.541**	0.9689	**34.93**	32.32
Big Model with Skip module (AccMotionDet + Eager $N_{\text{chk}}, W_{\text{clr}}$)	95.515	0.251	98.743	99.165	0.9731	34.37	**32.47**

5 Conclusion

We propose a lightweight, plug-and-play skip module $\mathcal{S}$ designed to accelerate real-time fire and smoke detection in indoor surveillance scenarios. Operating at a negligible fraction (≈4.5%) of the computational cost of the DL classifier $\mathcal{M}$, the skip module $\mathcal{S}$ efficiently filters out static background frames before they reach $\mathcal{M}$, reducing unnecessary inference overhead. Experiments on a real-world video dataset demonstrate that the proposed approach yields a throughput improvement of approximately 30% in frames per second while sustaining high detection recall above 94%.

Nevertheless, the proposed approach has certain limitations. First, the skip module $\mathcal{S}$ relies solely on inter-frame motion cues, which may be unsuitable for outdoor surveillance scenarios where persistent motion sources — such as swaying vegetation, moving vehicles, etc. — are continuously present, potentially causing $\mathcal{S}$ to pass the majority of frames to $\mathcal{M}$ and negating the efficiency gain. Second, the hyperparameters of $\mathcal{S}$ require dataset-specific tuning prior to deployment, incurring additional optimization overhead. Third, the overall detection accuracy of the system remains bounded by the capacity of $\mathcal{M}$, which is treated as a fixed component and not optimized in this work.

Future work may explore alternative skip strategies — such as leveraging color, texture, or learned features as gating cues — and extend the framework to outdoor surveillance scenarios where motion-based filtering is less effective. Additionally, designing a hyperparameter-free or self-adaptive skip module, as well as jointly optimizing the skip module and the classifier in an end-to-end training framework, represent promising directions for further improving system efficiency and detection accuracy.

References

[1] N. F. P. A. (NFPA), "NFPA report - largest fire losses in the united states." Jan. 1AD. Available: https://www.nfpa.org/education-and-research/research/nfpa-research/fire-statistical-reports/large-loss-fires-in-the-united-states/largest-fire-losses-in-the-united-states

[2] "South korea wildfires become biggest on record as disaster chief points to 'harsh reality' of climate crisis | south korea | the guardian." Jul. 2025. Available: https://www.theguardian.com/world/2025/mar/27/south-korea-fires-death-toll-rises-worst-in-history

[3] G. Cheng, X. Chen, C. Wang, X. Li, B. Xian, and H. Yu, "Visual fire detection using deep learning: A survey," *Neurocomputing*, vol. 596, p. 127975, 2024.

[4] D. Gragnaniello, A. Greco, C. Sansone, and B. Vento, "Fire and smoke detection from videos: A literature review under a novel taxonomy," *Expert Systems with Applications*, vol. 255, p. 124783, 2024.

[5] X. Geng, Y. Su, X. Cao, H. Li, and L. Liu, "YOLOFM: An improved fire and smoke object detection algorithm based on YOLOv5n," *Scientific Reports*, vol. 14, no. 1, p. 4543, 2024.

[6] Z. A. Khan *et al.*, "Optimized cross-module attention network and medium-scale dataset for effective fire detection," *Pattern Recognition*, vol. 161, p. 111273, 2025.

[7] R. A. Khan, U. I. Bajwa, R. H. Raza, and M. W. Anwar, "Beyond boundaries: Advancements in fire and smoke detection for indoor and outdoor surveillance feeds," *Engineering Applications of Artificial Intelligence*, vol. 142, p. 109855, 2025.

[8] Y. Cao, F. Yang, Q. Tang, and X. Lu, "An attention enhanced bidirectional LSTM for early forest fire smoke recognition," *IEEE Access*, vol. 7, pp. 154732–154742, 2019.

[9] M. M. Ali and M. Ghodrat, "Toward reliable fire detection in indoor CCTV footage: Reducing false alarms from benign flames using 3D attention CNNs," *IEEE Access*, 2025.

[10] P. V. A. de Venancio, R. J. Campos, T. M. Rezende, A. C. Lisboa, and A. V. Barbosa, "A hybrid method for fire detection based on spatial and temporal patterns," *Neural Computing and Applications*, vol. 35, no. 13, pp. 9349–9361, 2023.

[11] D. Gragnaniello, A. Greco, C. Sansone, and B. Vento, "FLAME: Fire detection in videos combining a deep neural network with a model-based motion analysis," *Neural Computing and Applications*, vol. 37, no. 8, pp. 6181–6197, 2025.

[12] A. Sobral and A. Vacavant, "A comprehensive review of background subtraction algorithms evaluated with synthetic and real videos," *Computer Vision and Image Understanding*, vol. 122, pp. 4–21, 2014.

[13] B. Garcia-Garcia, T. Bouwmans, and A. J. R. Silva, "Background subtraction in real applications: Challenges, current models and future directions," *Computer Science Review*, vol. 35, p. 100204, 2020.

[14] Z. Zivkovic and F. Van Der Heijden, "Efficient adaptive density estimation per image pixel for the task of background subtraction," *Pattern recognition letters*, vol. 27, no. 7, pp. 773–780, 2006.

[15] A. Ghodrati, B. E. Bejnordi, and A. Habibian, "Frameexit: Conditional early exiting for efficient video recognition," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 15608–15618.

[16] A. Jadon, M. Omama, A. Varshney, M. S. Ansari, and R. Sharma, "FireNet: A specialized lightweight fire & smoke detection model for real-time IoT applications," *arXiv preprint arXiv:1905.11922*, 2019.

[17] "Firesense." Available: https://www.firesense.eu/

[18] M. T. Cazzolato *et al.*, "Fismo: A compilation of datasets from emergency situations for fire and smoke analysis," in *Brazilian symposium on databases-SBBD*, SBC Uberlândia, Brazil, 2017, pp. 213–223.

[19] C. R. Steffens, R. N. Rodrigues, and S. S. da Costa Botelho, "An unconstrained dataset for non-stationary video based fire detection," in *2015 12th latin american robotics symposium and 2015 3rd brazilian symposium on robotics (LARS-SBR)*, IEEE, 2015, pp. 25–30.

[20] P. V. A. de Venâncio, A. C. Lisboa, and A. V. Barbosa, "An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices," *Neural Computing and Applications*, vol. 34, no. 18, pp. 15349–15368, 2022.

[21] E. Cetin, "Computer vision based fire detection software." 2015. Available: http://signal.ee.bilkent.edu.tr/VisiFire/

[22] B. C. Ko, S. J. Ham, and J. Y. Nam, "Modeling and formalization of fuzzy finite automata for detection of irregular fire flames," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 21, no. 12, pp. 1903–1912, 2011.

[23] P. Foggia, A. Saggese, and M. Vento, "Real-time fire detection for video-surveillance applications using a combination of experts based on color, shape, and motion," *IEEE TRANSACTIONS on circuits and systems for video technology*, vol. 25, no. 9, pp. 1545–1556, 2015.

[24] G. Lin, Y. Zhang, Q. Zhang, Y. Jia, G. Xu, and J. Wang, "Smoke detection in video sequences based on dynamic texture using volume local binary patterns." *KSII Trans. Internet Inf. Syst.*, vol. 11, no. 11, pp. 5522–5536, 2017.

[25] "AI-hub." Available: https://aihub.or.kr/aihubdata/data/view.do?dataSetSn=71751

[26] L. Huang, G. Liu, Y. Wang, H. Yuan, and T. Chen, "Fire detection in video surveillances using convolutional neural networks and wavelet transform," *Engineering Applications of Artificial Intelligence*, vol. 110, p. 104737, 2022.

[27] O. Önal and E. Dandıl, "Video dataset for the detection of safe and unsafe behaviours in workplaces," *Data in Brief*, vol. 56, p. 110791, 2024.

[28] D. Deniz, E. Ros, E. M. Ortigosa, and F. Barranco, "Optimized edge-cloud system for activity monitoring using knowledge distillation," *Electronics*, vol. 13, no. 23, p. 4786, 2024.

[29] M. Rohrbach *et al.*, "Recognizing fine-grained and composite activities using hand-centric features and script data," *International Journal of Computer Vision*, vol. 119, no. 3, pp. 346–373, 2016.

[30] R. Marroquin, J. Dubois, and C. Nicolle, "WiseNET: An indoor multi-camera multi-space dataset with contextual information and annotations for people detection and tracking," *Data in brief*, vol. 27, p. 104654, 2019.

[31] C. R. Steffens, R. N. Rodrigues, and S. S. da Costa Botelho, "Non-stationary VFD evaluation kit: Dataset and metrics to fuel video-based fire detection development," in *Latin american robotics symposium*, Springer, 2016, pp. 135–151.

[32] D. Mukhopadhyay, R. Iyer, S. Kadam, and R. Koli, "FPGA deployable fire detection model for real-time video surveillance systems using convolutional neural networks," in *2019 global conference for advancement in technology (GCAT)*, IEEE, 2019, pp. 1–7.

[33] G. Jocher, "YOLOv5 by Ultralytics." May 2020. doi: 10.5281/zenodo.3908559. Available: https://github.com/ultralytics/yolov5

[34] "Timm/hgnetv2_b5.ssld_stage2_ft_in1k · hugging face." Sep. 2025. Available: https://huggingface.co/timm/hgnetv2_b5.ssld_stage2_ft_in1k

[35] R. Wightman, "PyTorch image models," *GitHub repository*. https://github.com/rwightman/pytorch-image-models; GitHub, 2019. doi: 10.5281/zenodo.4414861

[36] C. Cui *et al.*, "Beyond self-supervision: A simple yet effective network distillation alternative to improve backbones," *arXiv preprint arXiv:2103.05959*, 2021.

[37] "PaddleClas/docs/en/models/PP-HGNetV2_en.md at f1233c18455b8acde4fc42ab0bea575fa06daa8e · PaddlePaddle/PaddleClas." Available: https://github.com/PaddlePaddle/PaddleClas/blob/f1233c18455b8acde4fc42ab0bea575fa06daa8e/docs/en/models/PP-HGNetV2_en.md?plain=1#L33

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[39] J. Hestness *et al.*, "Deep learning scaling is predictable, empirically," *arXiv preprint arXiv:1712.00409*, 2017.

[40] I. M. Alabdulmohsin, B. Neyshabur, and X. Zhai, "Revisiting neural scaling laws in language and vision," *Advances in Neural Information Processing Systems*, vol. 35, pp. 22300–22312, 2022.

[41] Y. Bahri, E. Dyer, J. Kaplan, J. Lee, and U. Sharma, "Explaining neural scaling laws," *Proceedings of the National Academy of Sciences*, vol. 121, no. 27, p. e2311878121, 2024.